

ARTIFICIAL INTELLIGENCE (AI) — Complete Roadmap & Interview Index

B.Tech CSE 4th Year • Basic → Intermediate → Advanced • Interview + Online Assessment Preparation

How to use this index: Follow the numbered sequence. Each topic is intentionally listed as a future chapter/lesson rather than explained here. The roadmap covers learning order, practical skills, interview depth, coding/OA relevance, projects, deployment, and revision.

A. Common Prerequisites & Interview Core

Purpose: Shared foundation before specializing.

A.1 Programming Fundamentals — Variables, data types, operators, input/output, conditions, loops, functions, scope and recursion.

A.2 Python for Technical Interviews — Lists, tuples, sets, dictionaries, strings, slicing, comprehensions, functions, modules and exceptions.

A.3 OOP Essentials — Classes, objects, inheritance, polymorphism, abstraction, encapsulation and composition.

A.4 DSA Foundation — Arrays, strings, linked lists, stacks, queues, hash tables, trees, heaps, graphs and recursion.

A.5 Algorithms — Sorting, searching, two pointers, sliding window, prefix sums, greedy, divide-and-conquer and dynamic programming.

A.6 Complexity — Big-O, time/space complexity, amortized thinking and trade-offs.

A.7 SQL Basics — SELECT, filtering, joins, grouping, subqueries, CTEs, window functions and query optimization basics.

A.8 Git & GitHub — Repositories, branching, commits, merge/rebase, pull requests and collaborative workflow.

A.9 Linux & CLI — Files, permissions, processes, pipes, grep, sed, awk, package management and environment variables.

A.10 Interview Communication — Explain approach, assumptions, complexity, trade-offs, debugging and project decisions.

1. AI Foundations

Purpose: Understand AI as the broad field and learn the major problem-solving paradigms.

1.1 What is AI? — Definitions, goals, intelligent agents, environments and real-world applications.

1.2 AI Problem Types — Classification, prediction, search, planning, recommendation, generation and decision-making.

1.3 Agents — Simple reflex, model-based, goal-based, utility-based and learning agents.

1.4 Search — State spaces, BFS, DFS, uniform-cost, greedy best-first and A*.

1.5 Heuristics — Admissibility, consistency and designing useful heuristics.

1.6 Game Playing — Minimax, alpha-beta pruning and evaluation functions.

1.7 Knowledge Representation — Logic, rules, ontologies, semantic networks and knowledge graphs.

1.8 Reasoning — Deduction, uncertainty, probabilistic reasoning and decision theory.

1.9 AI Ethics — Bias, fairness, privacy, transparency, safety, accountability and responsible AI.

2. Mathematics for AI

Purpose: Mathematical tools required to understand learning algorithms.

2.1 Linear Algebra — Vectors, matrices, dot products, norms, matrix multiplication, transpose, inverse and eigen concepts.

- 2.2 Probability** — Random variables, distributions, conditional probability, Bayes theorem and expectation.
- 2.3 Statistics** — Mean, variance, covariance, correlation, sampling and hypothesis concepts.
- 2.4 Calculus** — Derivatives, partial derivatives, gradients, chain rule and optimization intuition.
- 2.5 Optimization** — Loss functions, local/global minima, convexity intuition and gradient-based optimization.

3. Machine Learning for AI

Purpose: ML is the core learning engine behind modern AI systems.

- 3.1 ML Overview** — Supervised, unsupervised, semi-supervised and reinforcement learning.
- 3.2 Data Pipeline** — Collection, cleaning, splitting, preprocessing, features and labels.
- 3.3 Regression** — Linear regression, polynomial regression, regularization and evaluation.
- 3.4 Classification** — Logistic regression, k-NN, Naive Bayes, decision trees and SVM.
- 3.5 Ensembles** — Random forest, bagging, boosting, AdaBoost, gradient boosting and XGBoost concepts.
- 3.6 Unsupervised Learning** — K-means, hierarchical clustering, DBSCAN and dimensionality reduction.
- 3.7 Model Evaluation** — Confusion matrix, precision, recall, F1, ROC-AUC, PR-AUC and calibration.
- 3.8 Generalization** — Overfitting, underfitting, bias-variance, cross-validation and regularization.
- 3.9 Feature Engineering** — Encoding, scaling, selection, extraction and leakage prevention.

4. Deep Learning

Purpose: Neural-network foundations through modern architectures.

- 4.1 Neural Networks** — Perceptron, layers, activations, forward pass, loss and backpropagation.
- 4.2 Training** — Gradient descent, mini-batches, optimizers, initialization and learning rates.
- 4.3 Regularization** — Dropout, batch normalization, early stopping and weight decay.
- 4.4 CNNs** — Convolution, kernels, pooling, feature maps and image classification.
- 4.5 RNN Family** — RNN, vanishing gradients, LSTM and GRU.
- 4.6 Attention** — Queries, keys, values, self-attention and positional information.
- 4.7 Transformers** — Encoder, decoder, multi-head attention and transformer training.
- 4.8 Transfer Learning** — Pretrained models, fine-tuning, feature extraction and adapters.

5. Generative AI

Purpose: Modern AI generation, foundation models and LLM application patterns.

- 5.1 Generative Models** — Autoregressive models, autoencoders, VAEs, GANs and diffusion overview.
- 5.2 LLM Fundamentals** — Tokens, embeddings, context windows, transformer language models and next-token prediction.
- 5.3 Prompt Engineering** — Zero-shot, few-shot, role/context constraints, structured outputs and evaluation.
- 5.4 Embeddings** — Semantic vectors, similarity search and embedding-based retrieval.
- 5.5 RAG** — Chunking, embedding, retrieval, reranking, context construction and grounded generation.
- 5.6 Fine-Tuning** — Instruction tuning, parameter-efficient tuning and when fine-tuning is appropriate.
- 5.7 Agents** — Tools, planning, memory, workflows, guardrails and agent evaluation.
- 5.8 LLM Evaluation** — Correctness, relevance, groundedness, hallucination, latency and cost.
- 5.9 AI Safety** — Prompt injection, data leakage, model misuse, guardrails and human oversight.

6. AI Engineering

Purpose: Turn models into reliable software systems.

6.1 Python AI Stack — NumPy, pandas, scikit-learn and visualization ecosystem.

6.2 Deep Learning Frameworks — PyTorch/TensorFlow concepts, tensors, datasets, models and training loops.

6.3 APIs — REST, JSON, authentication, request/response design and model-serving endpoints.

6.4 Databases — Relational databases, NoSQL, vector databases and data-access patterns.

6.5 Deployment — Containers, model serving, cloud basics, inference APIs and environment management.

6.6 MLOps Basics — Experiment tracking, model registry, pipelines, monitoring and reproducibility.

6.7 Production Concerns — Latency, throughput, caching, batching, cost, reliability and security.

7. AI Projects & Portfolio

Purpose: Build evidence that you can apply concepts end-to-end.

7.1 Project 1 — Classical AI/ML — Problem definition, dataset, preprocessing, model, evaluation and API.

7.2 Project 2 — Deep Learning — Training pipeline, experiment tracking, error analysis and deployment.

7.3 Project 3 — GenAI/RAG — Document ingestion, retrieval, prompting, evaluation and production UI/API.

7.4 Project Documentation — Architecture, README, setup, results, limitations and future work.

7.5 Project Interview Defense — Why this model, alternatives rejected, metrics, failures, improvements and trade-offs.

8. AI Interview & OA Mastery

Purpose: Convert knowledge into interview performance.

8.1 AI Theory Questions — Definitions, algorithms, use cases, assumptions and limitations.

8.2 ML Rapid Fire — Bias-variance, regularization, leakage, metrics, cross-validation and model selection.

8.3 DL Rapid Fire — Backpropagation, activations, optimizers, CNN/RNN/transformer comparisons.

8.4 GenAI Rapid Fire — RAG vs fine-tuning, embeddings, hallucination, context and evaluation.

8.5 Coding OA — Arrays, strings, hashing, recursion, trees, graphs, greedy, DP and complexity.

8.6 SQL OA — Joins, aggregation, windows, CTEs, subqueries and query debugging.

8.7 Case Questions — Design an AI solution from requirements to deployment.

8.8 Mock Interviews — Timed explanation, coding, ML theory, project defense and behavioral rounds.

FINAL INTERVIEW / OA FLOW

Phase 1: Fundamentals → **Phase 2:** Core concepts → **Phase 3:** Coding & problem solving → **Phase 4:** Tools/frameworks → **Phase 5:** Projects → **Phase 6:** System/design thinking → **Phase 7:** Mock OAs → **Phase 8:** Technical + HR interviews → **Phase 9:** Revision and company-specific preparation.

Recommended answer depth: Definition → intuition → syntax/formula → small example → edge cases → complexity/trade-offs → real-world use → common interview questions.

End goal: Be able to explain concepts without notes, implement core algorithms/models, debug code, interpret results, discuss project decisions, and solve timed OA questions.